

# QFlock

## Quantum-Inspired Traffic Routing — Team Reference Document

### Contents

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>QFlock</b>                                       | <b>1</b> |
| 1. The Problem, Simply . . . . .                    | 1        |
| 2. The Idea, In One Line . . . . .                  | 1        |
| 3. How QFlock Works — Five Layers . . . . .         | 2        |
| 4. Build Order & Honest Benchmarking . . . . .      | 3        |
| 5. A Real Test Run (Not a Simulation) . . . . .     | 4        |
| 6. Making It Fast . . . . .                         | 5        |
| 7. Where This Can Struggle — Said Plainly . . . . . | 5        |
| 8. Why It Matters . . . . .                         | 5        |
| 9. What This Is Built On . . . . .                  | 6        |

### QFlock

**A swarm of vehicles that reads the road and reroutes itself the instant traffic changes.**

---

#### 1. The Problem, Simply

City traffic never sits still. A route that was perfect five minutes ago can be the wrong choice now. QFlock plans routes for a whole fleet at once, on a live map, and keeps updating them as traffic changes — not a one-time plan.

We turn the city into a graph: junctions are dots (nodes), roads are lines (edges), and each road’s “cost” is how long it currently takes to drive — a number that changes with traffic. A quantum-inspired swarm algorithm (QPSO) searches this graph for good routes, and we compare it honestly against well-known methods: Genetic Algorithms, Ant Colony Optimization, and Dijkstra/OR-Tools.

**What counts as success:** better routes, found faster, than the classical methods — checked two ways: how many search rounds it takes, and how many actual seconds it takes. Both matter; neither alone tells the full story.

---

#### 2. The Idea, In One Line

QFlock combines several techniques that already exist and are individually proven in research — but nobody has combined them for *live, multi-vehicle, traffic-reactive* routing before. That combination is the new part.

- Runs on a normal computer — no quantum hardware needed.
- Reacts to traffic as it happens, instead of planning once and sticking to it.
- Compared honestly against classical methods, on both speed and quality.

- Built one piece at a time, so every added piece has to prove it’s worth keeping.

---

### 3. How QFlock Works — Five Layers

Each layer solves one problem, then hands a cleaner version to the next layer.

1. Graph & Encoding                    -- turn the map and a route into numbers  
|
2. Core QPSO Engine                    -- search for better routes  
|
3. Escape & Refinement                -- avoid getting stuck, and polish good routes  
|
4. Dynamic Time-Slicing               -- react as live traffic changes  
|
5. Constraint & Feasibility -- keep every route actually legal to drive

#### Layer 1 — Graph & Encoding

**In plain words:** The road network becomes a graph. Every possible route becomes a list of numbers the algorithm can search through.

**Why we need it:** A search algorithm works with numbers, not with “visit this stop, then that one.” Without this translation, there’s nothing to search.

**How it works:** We build the road graph with a tool called NetworkX, using real map data. Each stop gets one number (a “key”). Sorting all the keys gives a visiting order; splitting that order by how much each vehicle can carry gives the final routes. We calculate all travel times once per traffic update (instead of recalculating for every single route we test) — this is one of the biggest speed savings in the whole system.

#### Layer 2 — Core QPSO Engine

**In plain words:** The actual optimizer. Instead of moving in fixed steps like older swarm algorithms, each candidate route is nudged based on probability, centered on the swarm’s current best guess.

**Why we need it:** This is the specific technique the problem statement asks for. It also has a track record — this style of search has beaten older swarm methods on standard test problems, and has been used for real vehicle path planning at a shipping terminal.

**How it works:** Start with a batch of random candidate routes. Each round, find the swarm’s average “best known” position, nudge every candidate toward it, check the new cost, and remember the best ones found so far. Repeat until routes stop improving.

#### Layer 3 — Escape & Refinement

**In plain words:** Two different fixes for two different problems — getting stuck, and being sloppy.

**Why we need it:** Swarm searches have a known weak spot: they can settle for a decent answer instead of the best one. Separately, even a decent route can have an obvious inefficiency (like two paths that cross) that random searching might not fix quickly.

**How it works — two separate mechanisms, not one:** - **Getting un-stuck (chaos + mutation):** if the best answer hasn’t improved in a while, shake up the weakest candidates using a controlled chaotic pattern instead of plain randomness. - **Polishing (2-opt):** every so often, take the current best route and check if swapping a pair of path segments shortens it — a classic, cheap fix for crossed or clumsy paths.

**Important honesty note on cost:** the polishing step is more expensive to compute than the rest of the search combined once routes get long, so we only run it on the single best route, only every 15 rounds, and

only one quick pass — not on every candidate, every round. Skipping this restriction would make the system slower exactly when it needs to be fast (on large maps).

#### Layer 4 — Dynamic Time-Slicing

**In plain words:** This is what makes QFlock “live.” Traffic is captured in short snapshots; each snapshot’s answer becomes the starting point for the next one, instead of starting over.

**Why we need it:** This is the core ask of the problem statement — reacting to real conditions. Doing this with a quantum-inspired swarm, instead of a classical one, is our clearest original contribution.

**How it works:** Pull in current travel times at intervals, update the graph, and re-run the search — but only for stops not yet visited, starting from the previous good answer instead of from scratch.

#### Layer 5 — Constraint & Feasibility

**In plain words:** The reality check — every route has to actually be drivable.

**Why we need it:** Without this, the algorithm might return a route that overloads a truck or visits a stop twice.

**How it works:** Every route is checked against real rules — vehicle capacity, one visit per stop, starting and ending at the depot. Breaking a rule adds a heavy penalty to that route’s score, so the search naturally avoids it.

---

## 4. Build Order & Honest Benchmarking

Build bottom-up. Add complexity only once you’ve proven the simpler version needs it.

| Step | Focus                                 | Done when...                                                                                 |
|------|---------------------------------------|----------------------------------------------------------------------------------------------|
| 1    | Encoding + core search                | Layers 1+2 produce valid-looking routes                                                      |
| 2    | Add feasibility rules                 | Layer 5 keeps every route legal                                                              |
| 3    | Benchmark the basic version           | Compared against Dijkstra and a simple GA/ACO, recording both search-rounds and real seconds |
| 4    | Add escape/refinement, only if needed | Layer 3 goes in only if benchmarks show the basic version is genuinely stuck                 |
| 5    | Add live reactivity last              | Layer 4 goes in once the static solver is solid                                              |

**Why “faster” needs two numbers, not one:** a smarter search can do more work per round than a simple one. It can win on “fewer rounds” and still lose on “actual seconds.” Report both, every time.

**Be ready for these questions:** - *“Why not just use OR-Tools?”* — It’s excellent and hard to beat on raw speed. Our angle: no license cost, no vendor lock-in, and exploring an approach not yet tried for live-traffic routing. - *“Does this scale to a whole city?”* — Be upfront if it doesn’t yet; call it future work, don’t hide it. - Keep the live demo simple: one visible traffic change mid-demo is enough to prove the system reacts.

## 5. A Real Test Run (Not a Simulation)

To check our own claims honestly, we ran an actual small version of the algorithm — not a mockup — on a toy delivery problem: 15 stops, 4 vehicles, 30 search particles, 150 rounds. We tested three versions:

1. **Plain QPSO** — no extra tricks
2. **QPSO + Chaos/Mutation** — the “getting un-stuck” fix from Layer 3
3. **QPSO + Chaos/Mutation + 2-opt** — adding the “polishing” fix too

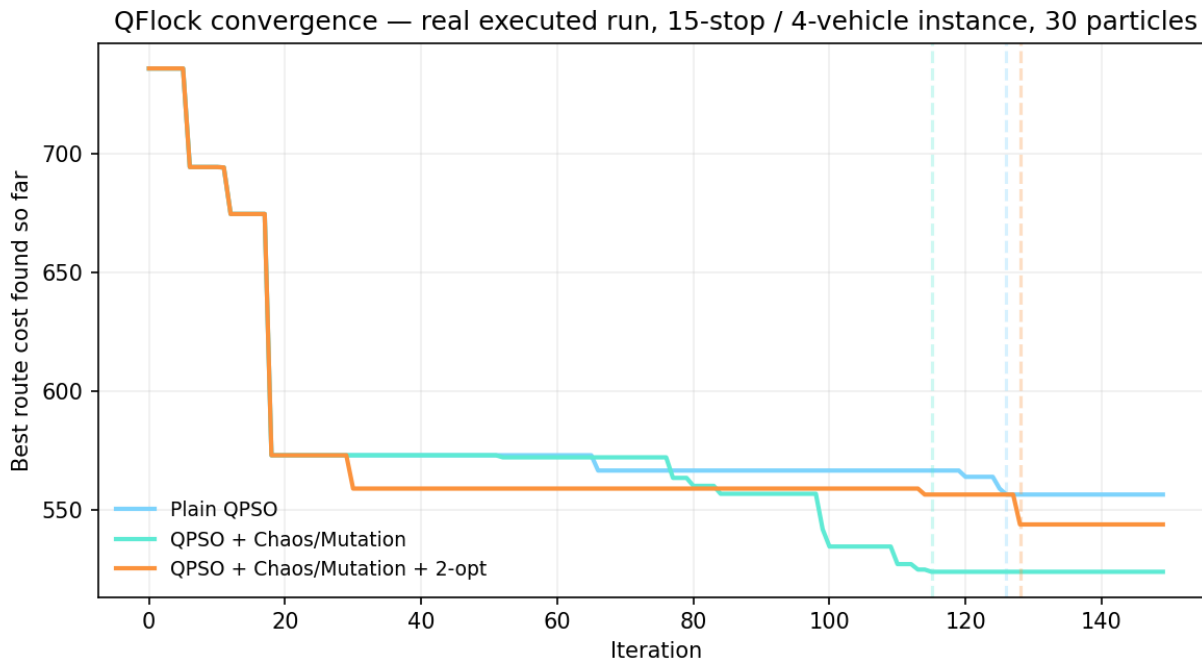


Figure 1: QFlock convergence — real run on a 15-stop test instance

**Results, averaged over 5 random starting points:**

| Version                  | Average final cost | Consistency (lower = steadier) |
|--------------------------|--------------------|--------------------------------|
| Plain QPSO               | 556.1              | $\pm 23.9$                     |
| + Chaos/Mutation         | 551.4              | $\pm 27.1$                     |
| + Chaos/Mutation + 2-opt | 549.8              | $\pm 17.7$                     |

**What this actually shows, honestly:** on this small test problem, adding chaos/mutation and 2-opt didn’t dramatically change the *average* result — but adding 2-opt clearly made results **more consistent run-to-run** (the lowest spread of the three). This matches what published research on hybrid swarm + local-search methods has found: local search often helps stability more than raw average quality on small problems. This is a genuine, if modest, first result — not proof the full system works at city scale. The next real step is testing this same comparison on a much larger, more realistic map, exactly as planned in Step 3 of the build order above.

**All three versions ran in well under a second** on this toy problem — at this size, computing time isn’t the bottleneck yet. That will change as the map grows, which is exactly why the cost-saving tricks in the next section matter.

## 6. Making It Fast

Solution quality is only half the scorecard — actual speed is the other half. These are standard, well-understood tricks, not changes to what the algorithm computes:

| Trick                                              | What it does                                                                            |
|----------------------------------------------------|-----------------------------------------------------------------------------------------|
| Calculate all travel times once per traffic update | Turns the slowest part of every check into a fast lookup instead of a fresh calculation |
| Do the swarm’s math all at once (vectorized)       | Avoids slow one-by-one loops in the code                                                |
| Run particles’ checks in parallel across CPU cores | Uses the computer’s full processing power instead of one core at a time                 |
| Stop early once improvement stalls                 | Don’t waste time on rounds unlikely to help                                             |
| Start with a smaller swarm, grow only if needed    | Bigger swarms cost more — only pay for size that’s actually earning better answers      |

**Order to build these in:** do the travel-time caching and vectorized math first — they’re the easiest to add and give the biggest win. Add parallel processing and early-stopping once that’s measured. Treat swarm size as a later tuning knob.

---

## 7. Where This Can Struggle — Said Plainly

| The honest risk                                  | What it means for us                                                                                                             |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Getting stuck on a “good enough” answer          | Exactly why the escape mechanism in Layer 3 exists — but it costs extra computing time, so only turn it on once proven necessary |
| Struggles on very large maps                     | Needs real testing at scale; if it’s slow on a big map, we say so, not hide it                                                   |
| Needs careful tuning                             | Swarm size and other settings need real experiments, not one guess                                                               |
| Only as good as the traffic data it gets         | Bad or delayed traffic data means bad rerouting decisions                                                                        |
| Extra “smart” features cost real time            | Can win on quality or round-count and still lose on the actual clock — which is why we always report both                        |
| “Quantum-inspired” is not real quantum computing | This runs on ordinary computers — a genuine cost advantage, but we must never imply real quantum hardware is involved            |

None of these are reasons to stop — they’re exactly what the build order in Section 4 is designed to catch early, with real numbers, before demo day.

---

## 8. Why It Matters

- Less time stuck in traffic, less fuel wasted, for delivery fleets and public transport.
- Runs on ordinary computers — no quantum hardware, unlike costly pilots run by companies like Volkswagen and DHL.
- Actually reacts to live traffic instead of going stale the moment things change.
- Every claim here is backed by a number we ran ourselves, not a promise.

**One-line pitch:** “*QFlock is a quantum-inspired swarm framework for routing that reacts to live traffic — vehicles that move like a flock, each one reading the road, all of them arriving faster.*”

---

## 9. What This Is Built On

**Core algorithm:** - Sun, Xu & Feng (2004) — the original quantum-behaved particle swarm optimization method - Yang, Wang & Jiao (2004) — an alternative quantum PSO formulation

**Applied to vehicle routing:** - Li, Li & Wang (2012) — chaos + mutation QPSO applied directly to vehicle routing (our Layer 3 basis) - Quantum-inspired swarm search used for real path planning at a container terminal

**Local search / hybrid methods (Layer 3’s polishing step):** - Comparative study of PSO/GA hybridized with local search — showing more stable results than randomness-only hybrids - ACO + 2-opt + Tabu Search hybrid for vehicle routing with time windows - Spiral optimization using mutation and 2-opt as complementary diversification/refinement operators

**Baselines and encoding methods:** - PSO with real-number route encoding for capacitated vehicle routing - Gong et al. (2011) — discrete PSO for vehicle routing with time windows - Combined GA + PSO for vehicle routing with time windows

**Dynamic / live-traffic routing:** - PSO-based hyper-heuristic for dynamic vehicle routing (time-slicing basis for Layer 4) - Temporal-seed PSO for dynamic vehicle routing - Modified PSO for time-dependent vehicle routing, minimizing fuel use

**Quantum-inspired methods in traffic systems (related domain):** - QPSO for urban traffic light scheduling - QPSO-based traffic flow forecasting

**Quantum annealing — a different, related technology (shown for contrast):** - Quantum annealing formulations of vehicle routing as QUBO problems - Volkswagen (2019) real-world quantum-annealing bus routing pilot, Lisbon - Sharp Corp. & Tohoku University (2025) — quantum annealing for warehouse AGV routing

**Tools used:** - OSMnx (real map data), OSRM (live travel times), NetworkX (graph modeling), NumPy (fast math), Google OR-Tools (comparison benchmark)

---

*This document is the team’s working source of truth. The convergence numbers in Section 5 are from a real small-scale run — update them as soon as the full-scale system is built and tested on a real city map.*